CanvasRead

Token-Efficient AI Scraper for Three.js & WebGL Websites

Product Requirements Document · v1.0 · Hackathon Submission
March 2026

1. Product Overview

CanvasRead is a token-efficient AI scraping pipeline that extracts structured data from Three.js and WebGL-rendered websites — surfaces that are completely opaque to conventional scrapers. It uses a three-layer architecture to minimise AI vision calls, then anchors every extracted dataset on-chain for verifiable provenance.

> **Core Insight:** Standard AI scraping burns thousands of tokens streaming every animation frame to a vision model. CanvasRead reduces this to near-zero by doing pixel-diff change detection in pure JS before any AI is ever invoked — cutting token usage by ~90%.

2. Problem Statement

2.1 The Scraping Dead Zone

The web is increasingly rendered through WebGL and Three.js. Product showcases, architectural visualisers, interactive portfolios, real estate 3D tours, and data-viz dashboards all live in this space. Their DOM is a single canvas element — there is no semantic structure, no accessible text nodes, no metadata for a crawler to parse.

Conventional scrapers return empty results. Headless browsers that screenshot and OCR the whole page are slow and expensive. Existing AI browsing agents that stream every frame to a vision model are prohibitively token-hungry for repeated or bulk use.

2.2 The Token Cost Problem

Running GPT-4V or Claude Vision on a Three.js site naively looks like this:

- Capture 30fps for 10 seconds = 300 frames
- Send all frames to vision model = 300 × ~800 tokens = 240,000 tokens per page
- At Claude Sonnet pricing, that is ~$0.72 per page load

CanvasRead's target: under 5,000 tokens per page — comparable to scraping a normal HTML site.

3. Solution Architecture

3.1 Three-Layer Pipeline

Layer	Name	What It Does	Token Cost
1	Change Detector	Pixel-diff hashing on canvas frames in pure JS/Playwright. Fires only when visual content meaningfully changes.	0 tokens
2	Selective Snap	Crops only the changed bounding-box region. Sends a single screenshot + tightly-constrained prompt to Claude Vision.	~200–800 tokens per snap
3	JSON Output	Merges all snap results into a clean, structured JSON document. Available for download or API consumption.	0 tokens

3.2 Layer 1 — Change Detection (Zero Tokens)

A Playwright headless browser loads the target URL and attaches a canvas observer. On each animation frame it computes a perceptual hash of the canvas buffer using a fast block-average algorithm. A new 'meaningful change' event is emitted only when:

- The hash distance from the previous snapshot exceeds a configurable threshold (default: 15% of pixels changed)
- The transition has settled — no further change detected for 300ms (debounce)
- The maximum cap of N snaps per page has not been reached (default: 12)

This filter eliminates idle animation loops, loading spinners, cursor trails, and particle effects that carry no semantic content.

3.3 Layer 2 — Selective Vision Snap (Minimal Tokens)

When a change event fires, the pipeline:

1. Computes the bounding box of changed pixels from the diff mask
2. Crops that region from the full canvas (Sharp/Canvas API)
3. Sends crop + a strict structured-output prompt to Claude Vision
4. Receives JSON-only response: { labels, values, descriptions, coordinates }
5. Merges into the running page-level extraction document

> **Prompt discipline:** The vision prompt explicitly forbids narrative description. It requests only structured JSON output with defined keys. This alone cuts average token response length by ~60% compared to open-ended prompts.

4. Core User Flows

4.1 Single Page Scrape

6. User pastes a Three.js URL into the CanvasRead web UI
7. Selects interaction mode: static, scroll-driven, or hover-driven
8. Pipeline runs — live dashboard shows snaps taken, tokens used, JSON building in real time
9. Extraction completes — user downloads JSON or copies to clipboard

4.2 Demo Mode (Hackathon Judges)

A curated set of 3 public Three.js sites is pre-loaded for one-click demos showing the token counter live alongside the extraction output.

5. Technical Stack

Layer	Technology	Rationale
Browser automation	Playwright (Node)	Canvas buffer access, headless Chromium, reliable cross-platform

Layer	Technology	Rationale
Pixel diff	Sharp + custom block-hash	Pure JS, no AI cost, sub-millisecond per frame
Vision AI	Claude Sonnet Vision	Best structured-output compliance, cost-efficient
Frontend	Next.js + Tailwind	Fast dev, good demo UX, SSR for the pipeline status page
Testing	TestSprite	Automated test generation for pipeline flows, no manual test code

6. TestSprite Integration

CanvasRead uses TestSprite as its primary QA layer in line with hackathon requirements. TestSprite is pointed at the two core flows:

- Scrape pipeline — valid URL in, structured JSON out, token counter increments correctly
- Edge cases — invalid URLs, Three.js sites with no meaningful state changes, oversized canvases

Because CanvasRead has well-defined input/output boundaries at each pipeline stage, TestSprite can generate a comprehensive test plan without any manual test code written.

> **Why this matters:** A scraper that silently returns incomplete data is worse than one that fails loudly. TestSprite's closed-loop diagnosis catches partial extraction bugs before submission.

7. Success Metrics

Metric	Target	Measurement
Token usage per Three.js page	< 5,000 tokens	Live counter in UI
Token reduction vs naive	> 85% fewer	Benchmarked against full-frame stream baseline
Extraction completeness	> 80% of visible labels captured	Manual spot-check on demo sites
Time to first extraction	< 30 seconds	Timed from URL submit

8. MVP Scope (Hackathon Weekend)

In Scope

- Change detection engine with configurable threshold
- Bounding-box crop + Claude Vision snap
- Structured JSON extraction output
- Next.js UI with live token counter and extraction preview
- TestSprite test suite on all core flows
- 3 pre-loaded demo Three.js sites

Out of Scope

- Authenticated / login-gated Three.js sites
- Multi-page crawling (single URL only for MVP)
- Custom extraction schemas (fixed JSON shape for MVP)
- Video or animated GIF output

9. Risks & Mitigations

Risk	Likelihood	Mitigation
Three.js site uses obfuscated canvas that defeats diff hashing	Medium	Fallback to full-page snap every 5s if no changes detected in 10s
Vision model misidentifies 3D labels as decorative	Low–Medium	Prompt includes few-shot examples of 3D UI text patterns
Playwright unable to access cross-origin canvas buffers	Low	Use screenshot API as fallback; most Three.js sites are same-origin

10. Hackathon Demo Narrative

The 60-second judge pitch:

> *"Try to scrape this Three.js product site. You get nothing."*
> *"Watch CanvasRead do it. 8 snaps. 3,200 tokens. Clean JSON."*

The live token counter is the hero element — it makes the efficiency story visceral and immediately legible to any judge. The side-by-side of a blank scraper result vs CanvasRead's structured JSON output tells the whole story in under a minute.